

Tuyển tập Bài tập: Kỹ thuật Phân rã & Quy đổi Bài toán

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Biên soạn: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 **Duc-Minh Vu**. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

Bảng Tổng hợp Bài tập Tuyển chọn

STT	Bài toán	Nền tảng	Độ khó	Kỹ thuật quy đổi & phân rã
1	Factory Machines	CSES 1620	★ Cơ bản	Tối ưu sang Quyết định (BS on Answer + Greedy Check)
2	Array Division	CSES 1085	★ Cơ bản	Chia mảng thành K phần $\rightarrow$ Chặt nhị phân tổng lớn nhất
3	Min Moves to Equal Elements II	LeetCode 462	★ Cơ bản	Quy đổi về Trung vị (Median) trên trục 1D
4	Best Meeting Point	LeetCode 296	★ Cơ bản	Phân rã 2D thành hai bài toán Trung vị 1D độc lập
5	Dist Max (Manhattan)	AtCoder ABC178E	★ Cơ bản	Xoay hệ tọa độ 45° đưa Manhattan về Chebyshev $O(N)$
6	Hamming Distance	LeetCode 461	★ Cơ bản	Phân rã độc lập từng bit $O(1)$
7	Sum of Subarray Minimums	LeetCode 907	★★ Trung cấp	Kỹ thuật tính đóng góp $A[i] \times (L_i \times R_i)$ trong $O(N)$
8	Xor Sum 4	AtCoder ABC147D	★★ Trung cấp	Độc lập từng bit: Đếm cặp bit (1, 0) đưa về $O(60N)$
9	Covered Points Count	Codeforces 1000C	★★ Trung cấp	Nén tọa độ + Mảng hiệu (Sweep-line Difference Array)
10	Solve The Maze	Codeforces 1365D	★★ Trung cấp	Quy đổi tính khả thi: Khóa ô xấu và BFS kiểm tra ô tốt
11	Subordinates	CSES 1674	★★ Trung cấp	Tree DP: Quy đổi cây thành tổng kích thước cây con

STT	Bài toán	Nền tảng	Độ khó	Kỹ thuật quy đổi & phân rã
12	Koko Eating Bananas	LeetCode 875	★★ Trung cấp	Chặt nhị phân tốc độ ăn tối thiểu (Optimization to Decision)
13	Max Median	Codeforces 1486D	★★★ Nâng cao	Chặt nhị phân đáp án + Nhị phân hóa mảng $\{-1, +1\}$
14	Coin Collector	CSES 1686	★★★ Nâng cao	Co cụm đồ thị (SCC Tarjan) → Quy về DP đường đi trên DAG
15	Camp Schedule	Codeforces 1137B	★★★ Nâng cao	Tối ưu số lần xuất hiện xâu → KMP Longest Border
16	LATGACH4 - Xếp gạch	VNOI LATGACH4	★★★ Nâng cao	Quy hoạch động Fibonacci $O(N)$ → Nhân ma trận $O(\log N)$



HƯỚNG DẪN GIẢI CHI TIẾT & GỢI Ý CHIẾN THUẬT

1. Nhóm Bài tập Cơ bản (6 bài)

Bài 1: [CSES 1620 - Factory Machines](#)

- **Đề bài tóm tắt:** Có N máy, máy thứ i mất k_i giây để làm ra một sản phẩm. Các máy làm việc song song độc lập. Tìm thời gian tối thiểu để hoàn thành T sản phẩm.
- **Phân tích & Thuật toán:**
- **Quy đổi từ Tối ưu sang Quyết định:** Thay vì tìm thời gian X , ta hỏi: "Trong thời gian X , các máy làm được tổng cộng bao nhiêu sản phẩm?"
- Hàm kiểm tra: $\text{products}(X) = \sum_{i=1}^N \lfloor X/k_i \rfloor$.
- Hàm này tăng đơn điệu theo X . Áp dụng **Chặt nhị phân trên tập nghiệm**:
 - Nếu $\text{products}(X) \geq T$: thời gian X khả thi, thử giảm X (`high = mid`).
 - Ngược lại: không đủ thời gian, tăng X (`low = mid + 1`).
- Độ phức tạp: $O(N \log(\text{MaxTime}))$.

Bài 2: [CSES 1085 - Array Division](#)

- **Đề bài tóm tắt:** Chia mảng N phần tử thành K đoạn con liên tiếp sao cho tổng lớn nhất của một đoạn là nhỏ nhất có thể.
- **Phân tích & Thuật toán:**
- Quy đổi sang bài toán quyết định bằng Chặt nhị phân: "Có thể chia mảng thành tối đa K đoạn sao cho mỗi đoạn có tổng $\leq S$ hay không?"
- Kiểm tra tính khả thi `check(S)` bằng thuật toán Tham lam trong $O(N)$: Duyệt từ trái sang phải, gom các phần tử vào đoạn hiện tại cho đến khi vượt quá S thì ngắt sang đoạn mới.

- Độ phức tạp: $O(N \log(\sum A_i))$.

Bài 3: [LeetCode 462 - Minimum Moves to Equal Array Elements II](#)

- **Đề bài tóm tắt:** Cho mảng N phần tử. Mỗi bước được tăng hoặc giảm 1 đơn vị của một phần tử. Tìm số bước tối thiểu để tất cả phần tử bằng nhau.
- **Phân tích & Thuật toán:**
- Cần tìm X tối thiểu hóa $\sum_{i=1}^N |A_i - X|$.
- **Quy đổi Toán học:** Giá trị tối ưu X luôn luôn là **Trung vị (Median)** của mảng A !
- Sắp xếp hoặc dùng `std::nth_element` tìm trung vị trong $O(N)$, sau đó tính tổng độ lệch trong $O(N)$.

Bài 4: [LeetCode 296 - Best Meeting Point](#)

- **Đề bài tóm tắt:** Cho lưới nhị phân $N \times M$, 1 đại diện cho nhà của một người bạn. Tìm vị trí gặp nhau có tổng khoảng cách Manhattan tới tất cả bạn bè là nhỏ nhất.
- **Phân tích & Thuật toán:**
- **Phân rã Độc lập Chiều không gian:**

$$\sum |x_i - X| + |y_i - Y| = \sum |x_i - X| + \sum |y_i - Y|$$

- Tách rời tọa độ x và tọa độ y của tất cả các ô số 1.
- Tìm trung vị của danh sách tọa độ x để có X_{opt} .
- Tìm trung vị của danh sách tọa độ y để có Y_{opt} .
- Độ phức tạp: $O(N \times M)$.

Bài 5: [AtCoder ABC178E - Dist Max](#)

- **Đề bài tóm tắt:** Cho N điểm trên mặt phẳng. Tìm khoảng cách Manhattan lớn nhất giữa hai điểm bất kỳ ($N \leq 2 \cdot 10^5$).
- **Phân tích & Thuật toán:**
- **Xoay hệ tọa độ 45° :** Đổi biến $(x_i, y_i) \rightarrow (u_i, v_i)$ với $u_i = x_i + y_i$ và $v_i = x_i - y_i$.
- Khoảng cách Manhattan chuyển thành khoảng cách Chebyshev:

$$\max_{i,j} (|x_i - x_j| + |y_i - y_j|) = \max(\max u - \min u, \max v - \min v)$$

- Duyệt một lượt tìm $\min$, $\max$ của u và v trong $O(N)$.

Bài 6: [LeetCode 461 - Hamming Distance](#)

- **Đề bài tóm tắt:** Đếm số vị trí bit khác nhau giữa hai số nguyên x và y .
- **Phân tích & Thuật toán:**
- Phân rã độc lập từng bit thông qua phép XOR: $z = x \oplus y$.
- Số bit khác nhau chính là số lượng bit 1 trong z : `__builtin_popcount(x ^ y)`.

2. Nhóm Bài tập Trung cấp (6 bài)

Bài 7: [LeetCode 907 - Sum of Subarray Minimums](#)

- **Đề bài tóm tắt:** Tính tổng các phần tử nhỏ nhất của mọi mảng con liên tiếp (modulo $10^9 + 7$).
 - **Phân tích & Thuật toán:**
 - Áp dụng **Kỹ thuật Tính Đóng góp**: Phần tử $A[i]$ đóng góp vào tổng bằng $A[i] \times (L_i \times R_i)$, trong đó L_i và R_i là khoảng cách tới phần tử nhỏ hơn gần nhất sang trái và phải.
 - Dùng **Monotone Stack** tính L_i và R_i trong $O(N)$.
-

Bài 8: [AtCoder ABC147D - Xor Sum 4](#)

- **Đề bài tóm tắt:** Cho mảng N phần tử ($N \leq 3 \cdot 10^5, A_i < 2^{60}$). Tính tổng $\sum_{i < j} (A_i \oplus A_j)$ (mod $10^9 + 7$).
 - **Phân tích & Thuật toán:**
 - **Phân rã Độc lập 60 Bit:**
 - Với mỗi bit $k \in [0, 59]$, đếm số lượng phần tử có bit k bật (C_1) và tắt ($C_0 = N - C_1$).
 - Số cặp có XOR bit k bằng 1 là $C_1 \times C_0$.
 - Đóng góp vào tổng là $(C_1 \times C_0 \pmod M) \times (2^k \pmod M)$.
 - Độ phức tạp: $O(60 \cdot N)$ tối ưu tuyệt đối.
-

Bài 9: [Codeforces 1000C - Covered Points Count](#)

- **Đề bài tóm tắt:** Cho N đoạn thẳng $[L_i, R_i]$ trên trục số ($L_i, R_i \leq 10^{18}$). Với mỗi $k \in [1, N]$, đếm số lượng điểm nguyên được phủ bởi đúng k đoạn.
 - **Phân tích & Thuật toán:**
 - Vì tọa độ lên tới 10^{18} , không thể dùng mảng hiệu trực tiếp.
 - **Quy đổi Tọa độ (Coordinate Compression / Event Points):**
 - Mỗi đoạn $[L, R]$ tạo ra hai sự kiện: bắt đầu tại L (+1) và kết thúc sau R (tại $R + 1, -1$).
 - Thu thập tất cả các điểm nút, sắp xếp lại.
 - Quét từ trái sang phải: giữa hai điểm sự kiện liên tiếp x_j và x_{j+1} , số đoạn phủ là cố định. Cộng độ dài $(x_{j+1} - x_j)$ vào kết quả của số lượng phủ hiện tại.
 - Độ phức tạp: $O(N \log N)$.
-

Bài 10: [Codeforces 1365D - Solve The Maze](#)

- **Đề bài tóm tắt:** Trong mê cung có người tốt **G** và kẻ xấu **B**. Bạn được phép xây tường **#** tại các ô trống để ngăn tất cả **B** không thoát được ra cửa (N, M) nhưng tất cả **G** vẫn phải thoát ra được.
- **Phân tích & Thuật toán:**
- **Chiến lược Tham lam & Quy đổi Bất biến:**
 - Để ngăn chặn **B**, cách tốt nhất là **xây tường xung quanh tất cả các ô kề của mọi kẻ xấu B**.

- Nếu một ô kề của **B** là người tốt **G**, lập tức vô nghiệm (No).
 - Sau khi xây tường xung quanh mọi **B**: Chạy BFS từ lối thoát (N, M) ngược vào trong mê cung.
 - Kiểm tra xem có kẻ xấu **B** nào đi tới được lối thoát không.
 - Kiểm tra xem tất cả người tốt **G** có đi tới được lối thoát không.
 - Độ phức tạp: $O(N \times M)$.
-

Bài 11: [CSES 1674 - Subordinates](#)

- **Đề bài tóm tắt:** Trong một công ty có cấu trúc cây phân cấp, đếm số cấp dưới của mỗi nhân viên.
- **Phân tích & Thuật toán:**
- **Quy đổi Cấu trúc Cây:** Số cấp dưới của nút u chính bằng **Kích thước cây con** trừ đi chính nó:

$$\text{size}[u] = 1 + \sum_{v \in \text{children}(u)} \text{size}[v]$$

- DFS duyệt hậu thứ tự (Post-order Traversal) trong $O(N)$.
-

Bài 12: [LeetCode 875 - Koko Eating Bananas](#)

- **Đề bài tóm tắt:** Koko có H giờ để ăn hết N nải chuối. Tìm tốc độ ăn chuối tối thiểu K (quả/giờ) để ăn hết toàn bộ chuối trong H giờ.
 - **Phân tích & Thuật toán:**
 - Chặt nhị phân tốc độ $K \in [1, \max(\text{piles})]$.
 - Với mỗi K , tổng thời gian cần là: $\text{hours}(K) = \sum \lceil \text{pile}_i / K \rceil$.
 - Nếu $\text{hours}(K) \leq H \implies$ Tốc độ K khả thi.
 - Độ phức tạp: $O(N \log(\max A))$.
-

3. Nhóm Bài tập Nâng cao (4 bài)

Bài 13: [Codeforces 1486D - Max Median](#)

- **Đề bài tóm tắt:** Cho mảng N phần tử và số K . Tìm trung vị lớn nhất có thể của một đoạn con liên tiếp có độ dài $\geq K$.
 - **Phân tích & Thuật toán:**
 - **Chặt nhị phân trên đáp án X :**
 - Biến đổi mảng A thành mảng B gồm $\{-1, +1\}$: $B_i = +1$ nếu $A_i \geq X$, ngược lại $B_i = -1$.
 - Trung vị của đoạn $\geq X \iff$ Tổng của đoạn trên mảng $B \geq 1$.
 - Kiểm tra xem có đoạn nào độ dài $\geq K$ có tổng > 0 hay không bằng Mảng tiền tố và duy trì giá trị nhỏ nhất của tiền tố phía trước trong $O(N)$.
 - Độ phức tạp: $O(N \log N)$ cực kỳ xuất sắc!
-

Bài 14: [CSES 1686 - Coin Collector](#)

- **Đề bài tóm tắt:** Trên đồ thị có hướng, mỗi phòng có số đồng xu. Bạn có thể thu thập đồng xu khi đi vào phòng. Tìm số đồng xu tối đa có thể thu thập được trên một đường đi hợp lệ.
 - **Phân tích & Thuật toán:**
 - **Quy đổi Đồ thị bằng Thuật toán Co Cùm (SCC Contraction):**
 - Nếu có một chu trình, bạn có thể thu thập toàn bộ xu của mọi đỉnh trong chu trình đó.
 - Tìm các thành phần liên thông mạnh (SCC) bằng Tarjan hoặc Kosaraju.
 - Co mỗi SCC thành một siêu đỉnh (Super-node) với trọng số bằng tổng xu của toàn bộ SCC.
 - Đồ thị mới là một **Đồ thị có hướng không chu trình (DAG)**!
 - Bài toán quy về: **Tìm đường đi có tổng trọng số lớn nhất trên DAG** (giải bằng Quy hoạch động / Topological Sort trong $O(V + E)$).
 - Độ phức tạp: $O(V + E)$.
-

Bài 15: [Codeforces 1137B - Camp Schedule](#)

- **Đề bài tóm tắt:** Cho xâu nhị phân S và T . Hoán vị lại các ký tự của S để số lần xuất hiện của T là lớn nhất.
 - **Phân tích & Thuật toán:**
 - **Quy đổi về KMP Longest Border:**
 - Để tối đa hóa số lần xuất hiện của T , sau khi đặt xong một lần xâu T , các lần tiếp theo chỉ cần nối thêm phần đuôi của T chồng chập với tiền tố dài nhất của T (Border của T).
 - Dùng mảng π của thuật toán KMP tìm tiền tố vừa là hậu tố dài nhất của T .
 - Tham lam ghép xâu T ban đầu, rồi liên tục ghép đoạn đuôi $T[\pi[|T|] \dots |T| - 1]$ chừng nào số lượng ký tự 0 và 1 trong S vẫn còn đủ.
 - Độ phức tạp: $O(|S| + |T|)$.
-

Bài 16: [VNOI - LATGACH4 \(Xếp gạch\)](#)

- **Đề bài tóm tắt:** Có bao nhiêu cách lát thanh chữ nhật kích thước $2 \times N$ bằng các viên gạch 1×2 và 2×1 ($N \leq 10^9$)?
- **Phân tích & Thuật toán:**
- Công thức truy hồi: $F(N) = F(N - 1) + F(N - 2)$ (Dãy số Fibonacci).
- Với $N \leq 10^9$, quy hoạch động tuyến tính $O(N)$ dính TLE.
- **Quy đổi sang Lũy thừa Ma trận (Matrix Exponentiation):**

$$\begin{pmatrix} F(N+1) \\ F(N) \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^N \begin{pmatrix} F(1) \\ F(0) \end{pmatrix}$$

- Tính lũy thừa ma trận nhị phân trong $O(2^3 \log N)$ thời gian.
-



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.